COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/30/2026	**DATE SUBMITTED:**	09/30/2026
NAME:	Milan, Kherwin Moradillo, Eunice, Nieles,Cenndy Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Seatwork 10.1 Graph

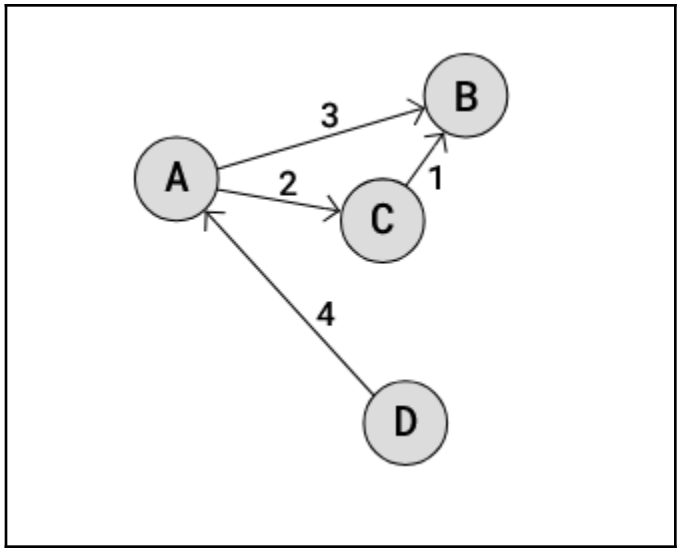
1. PROCEDURE OUTPUT

What procedures did you perform?

1. Define the following: (10)	
GRAPH	A graph is a non-linear data structure written as G = (V, E), where V is a finite set of vertices and E is a set of edges connecting pairs of vertices. Graphs model relationships between objects, such as roads between cities, links between web pages, or connections in a network.
VERTEX	A single point or element in a graph, also called a node. It represents an object such as a city, a person, or a device.
NODES	a node (or vertex) is a basic unit that represents an object, entity, or point of interest within a graph. Nodes serve as the building blocks of graphs, and their primary purpose is to model the elements of a system or network.
WEIGHT	A weight is a numerical value attached to an edge. It represents the cost of using that edge, such as distance, time, price, or capacity. A graph whose edges have weights

	is called a weighted graph. A graph without weights is unweighted, and every edge is treated as equal.
PATH	A sequence of vertices in which each consecutive pair is joined by an edge.
DIRECTED GRAPH	A directed graph is a graph in which every edge has a direction, drawn as an arrow from a source (tail) vertex to a destination (head) vertex. An edge (A, B) means you can travel from A to B, but not from B to A unless a separate edge (B, A) exists.
CONNECTED GRAPH	A connected graph is an undirected graph in which there is a path between every pair of vertices, so nothing is isolated. If some vertices cannot be reached from others, the graph is disconnected.
DIRECTED CYCLIC GRAPH	A directed cyclic graph is a directed graph that contains at least one cycle, a path that starts and ends at the same vertex, following the edge directions
ADJACENCY MATRIX	Adjacency Matrix is a square matrix used to represent a finite graph. The elements of the matrix indicate whether pairs of vertices are adjacent or not in the graph. An adjacency matrix is a simple and straightforward way to represent graphs and is particularly useful for dense graphs.
ADJACENCY LIST	An adjacency list represents a graph as an array (or list) of V lists, where the list at index i holds all the vertices adjacent to vertex i (with weights, if the graph is weighted).

2. Identify the parts of the following graph. (10)



DIRECTED GRAPH - It is a directed graph because the arrowhead is pointed out on each edge on one end, which means you can only travel in that direction. It is weighted because the arrows have numbers hahah	
VERTICES	A, B, C, D (4 NODES)
EDGES	A - B, A - C, C - B, D - A (4 EDGES/CONNECTING ARROWS)
WEIGHTS	3, 2, 1 ,4 A -B = 3, A - C = 2, C - B = 1, D - A = 4

3. Plot the adjacency matrix of the graph above. (10 pts)

Weighted Matrix

$$E = \begin{bmatrix} 0 & 3 & 2 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

[0 1 0 0] [4 0 0 0]

Vertex/ Nodes: A, B, C, D	
GRAPH EDGE	**MATRIX**
A - B, weight 3	E = [A][B] = 3
A - C, weight 2	E = [A][C] = 2
C - B, weight 1	E = [C][B] = 1
D - A, weight 4	E = [D][A] = 4

Unweighted Matrix

E = [0 1 1 0] [0 0 0 0] [0 1 0 0] [1 0 0 0]

Vertex/ Nodes: A, B, C, D (the weights will become 1)		
GRAPH EDGE	**WEIGHTED MATRIX**	**UNWEIGHTED MATRIX**
A - B, weight 3	E = [A][B] = 3	E = [A][B] = 1
A - C, weight 2	E = [A][C] = 2	E = [A][C] = 1

C - B, weight 1	E = [C][B] = 1	E = [C][B] = 1
D - A, weight 4	E = [D][A] = 4	E = [D][A] = 1

4. Create a program in C++ program to create a graph that looks like in the above figure.

Program

main.cpp

AsyncAct > main.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <utility>

class Graph {
    int V;
    std::vector<std::vector<std::pair<int,int>>> adj;
public:
    Graph(int v) : V(v), adj(v) {}

    void addEdge(int from, int to, int weight) {
        adj[from].push_back({to, weight});
    }

    void printAdjList() {
        std::cout << "Adjacency List:\n";
        for (int i = 0; i < V; i++) {
            std::cout << char('A' + i) << " -> ";
            for (auto &e : adj[i])
                std::cout << char('A' + e.first) << "(" << e.second << ") ";
            std::cout << "\n";
        }
    }

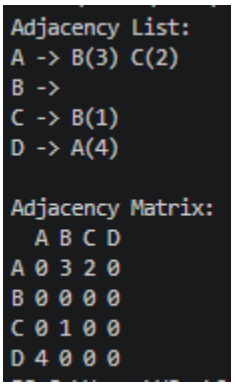
    void printAdjMatrix() {
        std::vector<std::vector<int>> m(V, std::vector<int>(V, 0));
        for (int i = 0; i < V; i++)
            for (auto &e : adj[i])
                m[i][e.first] = e.second;

        std::cout << "\nAdjacency Matrix:\n  ";
        for (int i = 0; i < V; i++) std::cout << char('A' + i) << " ";
        std::cout << "\n";
        for (int i = 0; i < V; i++) {
            std::cout << char('A' + i) << " ";
            for (int j = 0; j < V; j++) std::cout << m[i][j] << " ";
            std::cout << "\n";
        }
    }
};

int main() {
    Graph g(4);

    g.addEdge(0, 1, 3);
    g.addEdge(0, 2, 2);
    g.addEdge(2, 1, 1);
    g.addEdge(3, 0, 4);

    g.printAdjList();
    g.printAdjMatrix();
    return 0;
}
```

Output


5. Differentiate a BFS vs DFS types of search.

	Breadth-First Search (BFS)	Depth-First Search (DFS)
ORDER	Explores all neighbors at the current level before going deeper	Goes as deep as possible along one branch, then backtracks
TRAVERSAL	Level by level	Branch by branch
TIME COMPLEXITY	O(V + E)	O(V + E)
SPACE	Can be large, since it stores a whole level of nodes	Usually smaller, proportional to the depth of the graph
STRUCTURE	Queue (FIFO)	Stack (LIFO) or recursion
PATH	Finds the shortest path in an unweighted graph	Does not guarantee the shortest path

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Main

```cpp
#include <iostream>
#include "BST.h"

int main(){

    //create the root node
    BinarySearchTree<int> tree(20);

    tree.insert(10);
    tree.insert(5);
    tree.insert(70);
    tree.insert(60);
    tree.insert(80);
    tree.insert(50);


    std::cout << "PreOrder : ";
    tree.preOrder();
    std::cout << std::endl;

    std::cout << "InOrder : ";
    tree.InOrder();
    std::cout << std::endl;

    std::cout <<"PostOrder: " ;
    tree.PostOrder();
    std::cout << std::endl;



    return 0;
}
```

Bst h

```
#ifndef BST_H
#define BST_H

template<typename T>
class BinarySearchTree{
    private:
    //create a key
    T key;
    //pointers for left and right
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;


    public:
    //constructor
    BinarySearchTree(T value);

    //insert left and eight
    void insert(T value);
    void preOrder();
    void InOrder();
    void PostOrder();



    //traversing


    //searching
};

template <typename T>
BinarySearchTree<T>::BinarySearchTree (T value){
    key = value;
    left = nullptr;
    right = nullptr;
}
```

```cpp
template<typename T>
void BinarySearchTree<T>::insert(T value){
    //left or right


    //go left if value < key inset to left
    if(value < key){
        if (left == nullptr){
        left = new BinarySearchTree<T>(value);
        }else{
            left -> insert(value);
        }
    }else if(value > key) {
        if (right == nullptr) {
            right = new BinarySearchTree<T>(value);
        }
    else {
        right->insert(value);
        }

    }
}
template <typename T>
void BinarySearchTree<T>::preOrder(){

    std::cout<<key << " ";
    if (left != nullptr){
        left -> preOrder();
    }

    if(right != nullptr){
        right -> preOrder();
    }
}
```

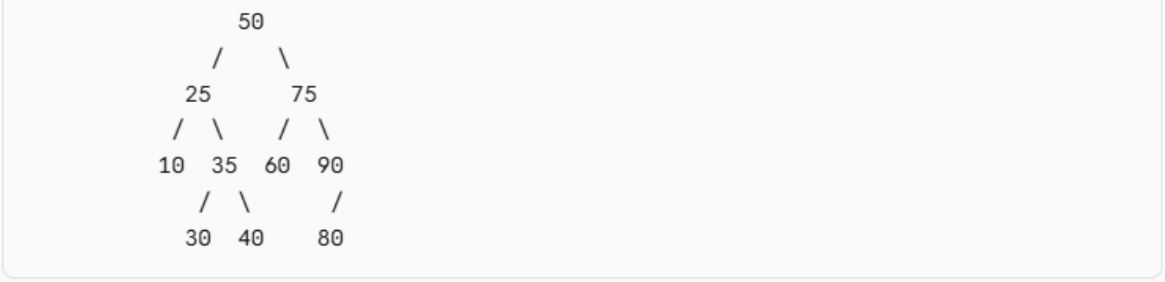
```
template <typename T>
void BinarySearchTree<T>::InOrder(){
    //go left
    if (left != nullptr){
        left -> InOrder();
    }
    std::cout<< key << " ";
    //go right
    if (right != nullptr){
       right -> InOrder();
    }
}

template<typename T>
void BinarySearchTree<T>::PostOrder(){
    //go left
    if (left != nullptr){
        left -> PostOrder();
    }
    //go right
    if (right != nullptr){
       right -> PostOrder();
    }
    std::cout<< key << " ";
}



#endif
```

Sequence 1: [50, 25, 75, 10, 35, 60, 90, 30, 40, 80]



- **In-order:** 10, 25, 30, 35, 40, 50, 60, 75, 80, 90
- **Pre-order:** 50, 25, 10, 35, 30, 40, 75, 60, 90, 80
- **Post-order:** 10, 30, 40, 35, 25, 60, 80, 90, 75, 50

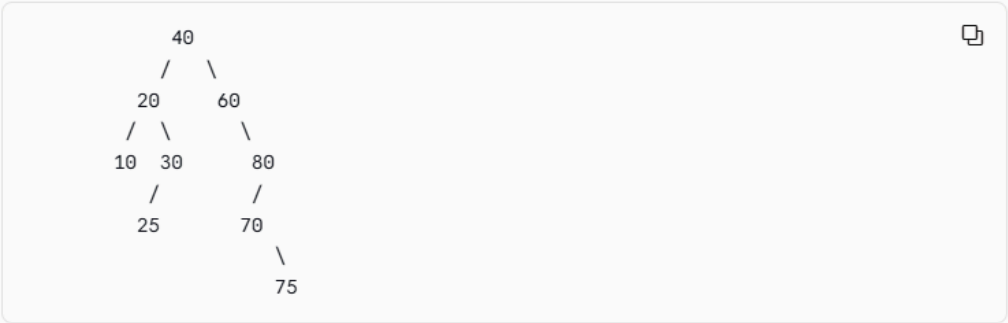
Sequence 2: [K, E, P, B, H, M, T, A, S]



- **In-order:** A, B, E, H, K, M, P, S, T
- **Pre-order:** K, E, B, A, H, P, M, T, S
- **Post-order:** A, B, H, E, M, S, T, P, K

- **Post-order:** A, B, H, E, M, S, T, P, K

Sequence 3: [40, 20, 10, 30, 25, 60, 80, 70, 75]



- **In-order:** 10, 20, 25, 30, 40, 60, 70, 75, 80
- **Pre-order:** 40, 20, 10, 30, 25, 60, 80, 70, 75
- **Post-order:** 10, 25, 30, 20, 75, 70, 80, 60, 40

Sequence 5: [g, C, L, a, E, i, N, d, J]

I treated the comparisons as case-insensitive alphabetical order (a < c < d < e < g < i < j < l < n) — worth double-checking with your instructor, since if it's meant to be strict ASCII order (where all uppercase letters sort before all lowercase letters), the tree comes out differently.



- **In-order:** a, C, d, E, g, i, J, L, N
- **Pre-order:** g, C, a, E, d, L, i, J, N
- **Post-order:** a, d, E, C, J, i, N, L, g

A couple of tips for drawing these by hand: work through the insertions one at a time rather than trying to place the whole tree at once, and for the traversals, remember the pattern — **in-order** = left → root → right (gives sorted order for numbers), **pre-order** = root → left → right, **post-order** = left → right → root.

main

```cpp
#include <iostream>
#include <cctype>
#include "BST.h"

int main(){

    //Sequence 1
    BinarySearchTree<int> tree1(50);
    tree1.insert(25);
    tree1.insert(75);
    tree1.insert(10);
    tree1.insert(35);
    tree1.insert(60);
    tree1.insert(90);
    tree1.insert(30);
    tree1.insert(40);
    tree1.insert(80);

    std::cout << "Sequence 1" << std::endl;
    std::cout << "PreOrder : ";
    tree1.preOrder();
    std::cout << std::endl;

    std::cout << "InOrder : ";
    tree1.InOrder();
    std::cout << std::endl;

    std::cout << "PostOrder: ";
    tree1.PostOrder();
    std::cout << std::endl << std::endl;


    //Sequence 2
    BinarySearchTree<char> tree2('K');
    tree2.insert('E');
    tree2.insert('P');
```

```
    tree2.insert('B');
    tree2.insert('H');
    tree2.insert('M');
    tree2.insert('T');
    tree2.insert('A');
    tree2.insert('S');

    std::cout << "Sequence 2" << std::endl;
    std::cout << "PreOrder : ";
    tree2.preOrder();
    std::cout << std::endl;

    std::cout << "InOrder : ";
    tree2.InOrder();
    std::cout << std::endl;

    std::cout << "PostOrder: ";
    tree2.PostOrder();
    std::cout << std::endl << std::endl;


    //Sequence 3
    BinarySearchTree<int> tree3(40);
    tree3.insert(20);
    tree3.insert(10);
    tree3.insert(30);
    tree3.insert(25);
    tree3.insert(60);
    tree3.insert(80);
    tree3.insert(70);
    tree3.insert(75);

    std::cout << "Sequence 3" << std::endl;
    std::cout << "PreOrder : ";
    tree3.preOrder();
    std::cout << std::endl;

    std::cout << "InOrder : ";
    tree3.InOrder();
```

```
    std::cout << std::endl;

    std::cout << "PostOrder: ";
    tree3.PostOrder();
    std::cout << std::endl << std::endl;


    //Sequence 4
    BinarySearchTree<int> tree4(30);
    tree4.insert(15);
    tree4.insert(50);
    tree4.insert(10);
    tree4.insert(20);
    tree4.insert(40);
    tree4.insert(60);
    tree4.insert(35);
    tree4.insert(45);
    tree4.insert(55);

    std::cout << "Sequence 4" << std::endl;
    std::cout << "PreOrder : ";
    tree4.preOrder();
    std::cout << std::endl;

    std::cout << "InOrder : ";
    tree4.InOrder();
    std::cout << std::endl;

    std::cout << "PostOrder: ";
    tree4.PostOrder();
    std::cout << std::endl << std::endl;


    //Sequence 5
    //lowercased so letters compare alphabetically (a < c < d ... )
instead of by raw ASCII
    BinarySearchTree<char> tree5((char)std::tolower('g'));
    tree5.insert((char)std::tolower('C'));
    tree5.insert((char)std::tolower('L'));
```

```
    tree5.insert((char)std::tolower('a'));
    tree5.insert((char)std::tolower('E'));
    tree5.insert((char)std::tolower('i'));
    tree5.insert((char)std::tolower('N'));
    tree5.insert((char)std::tolower('d'));
    tree5.insert((char)std::tolower('J'));

    std::cout << "Sequence 5" << std::endl;
    std::cout << "PreOrder : ";
    tree5.preOrder();
    std::cout << std::endl;

    std::cout << "InOrder : ";
    tree5.InOrder();
    std::cout << std::endl;

    std::cout << "PostOrder: ";
    tree5.PostOrder();
    std::cout << std::endl;


    return 0;
}
```